

Lecture 06

- Arbitrary-sized data
 - All my favorite hockey teams (L06)
 - All the eggs falling around Mario (L07)
 - All the balls bouncing around the box (Pset 4)
 - ...
- Self-referential type comments
- Naturally recursive functions
- Sound theory makes it easy to understand functions that might otherwise be confusing

```
(@htdd ListOfString)
```

```
:: ListOfString is one of:
```

```
:: - empty
```

```
:: - (cons String ListOfString)
```

```
(define (fn-for-los los)
```

```
  (cond [(empty? los) (...)]
```

```
        [else
```

```
          (... (first los)
```

```
                (fn-for-los (rest los)))]))
```

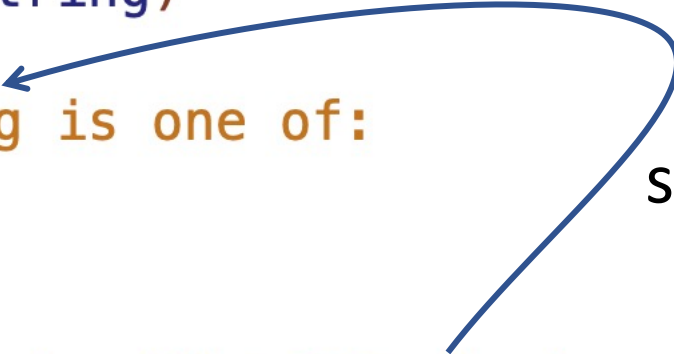
```
(@htdd ListOfString)
```

```
;; ListOfString is one of:
```

```
;; - empty
```

```
;; - (cons String ListOfString)
```

self reference



```
(define (fn-for-los los)
```

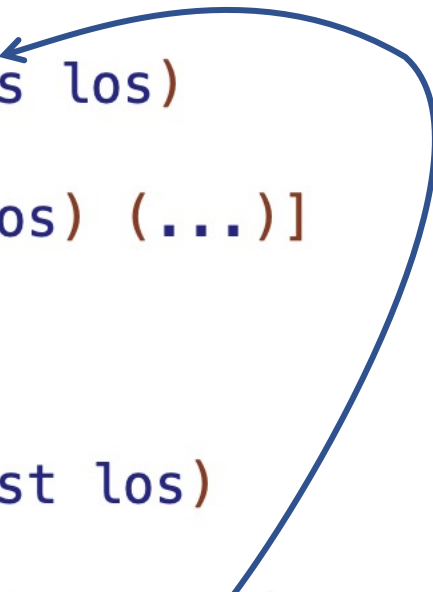
```
  (cond [(empty? los) (...)]
```

```
        [else
```

```
          (... (first los)
```

```
                (fn-for-los (rest los)))]))
```

natural recursion



```
(@template  
  (define (contains-canucks? los)  
    (cond [(empty? los) (...)]  
          [else  
            (... (first los)  
                  (contains-canucks? (rest los)))])))
```

Trusting the Natural Recursion

result of natural recursion (RNR) will be correct if and only if

- correct base case result
- correct contribution of first
- correct combination of contribution and RNR

(@template

```
(define (contains-canucks? los)
```

```
(cond [(empty? los) (...)]
```

```
[else
```

```
(... (first los)
```

```
(contains-canucks? (rest los))))))
```

base case result

correct result for empty

combination

how to combine (first los) and
result of natural recursion

arbitrary-sized information -> requires well-formed self-referential data definition

```
(@htdd ListOfString)
```

```
;; ListOfString is one of:
```

```
;; - empty
```

```
;; - (cons String ListOfString)
```

one of with:

one or more base subclass

one or more self-reference subclasses

arbitrary-sized information -> requires self-referential data definition

```
(@htdf contains-canucks?)  
(@signature ListOfString -> Boolean)  
;; produce true if los contains "Canucks"  
(check-expect (contains-canucks? empty) false)  
(check-expect (contains-canucks? (cons "Canucks" (cons "Flames" empty))) true)  
(check-expect (contains-canucks? (cons "Flames" (cons "Canucks" empty))) true)  
(check-expect (contains-canucks? (cons "Flames" (cons "Leafs" empty))) false)
```

test base case first

```
;(define (contains-canucks? los) false)
```

test 2 long

test recursion on both sides of conditional

```
(@template-origin ListOfString)
```

```
(@template  
  (define (contains-canucks? los)  
    (cond [(empty? los) (...)]  
          [else  
            (... (first los)  
                  (contains-canucks? (rest los)))])))
```

rename natural recursion when templating

```
(define (contains-canucks? los)  
  (cond [(empty? los) false]  
        [else  
          (if (string=? (first los) "Canucks")  
              true  
              (contains-canucks? (rest los)))]))
```

can "trust the natural recursion" if and only if:

- correct base case result
- correct contribution of first
- correct combination

Function	Base case result	Combination

Function	Base case result	Combination
sum	0	(+ <first> RNR)
product	1	(* <first> RNR)
count	0	(+ 1 RNR)
doubles	empty	(cons (* 2 <first>) RNR)